The invisible infrastructure of video game preservation

The global effort to preserve video game software depends on an ecosystem of community-built tools, metadata standards, and volunteer organizations that together constitute one of the most sophisticated grassroots digital preservation systems ever constructed. At the center of this ecosystem sits an unassuming XML file format -- the DAT file -- that functions as a canonical manifest linking every known ROM dump or disc image to its cryptographic fingerprint. This format, created by a single British developer in 2008 and hosted on a website that went dark around 2010, now underpins the verification of hundreds of thousands of software artifacts across dozens of organizations. Remarkably, the practices these communities have invented independently mirror -- and in some respects surpass -- the formal digital preservation standards used by institutions like the Library of Congress.

This report traces the history, technical architecture, and organizational landscape of ROM and disc image preservation, from the DAT file formats born in the late-1990s MAME community through the major preservation groups operating today, the ROM manager tools that consume their metadata, and the deep structural parallels with institutional digital preservation.

Part 1: The DAT file -- why it exists

The problem that never ends: all media is dying

Every medium that stores digital data is in the process of losing it.

Magnetic domains on spinning hard drive platters gradually lose their orientation as thermal energy randomizes them -- a process physicists call superparamagnetic decay. Flash memory cells in SSDs leak stored charge through their oxide barriers at rates that accelerate with heat and age; an unpowered SSD in a warm closet can begin losing data within months. The aluminum and polycarbonate layers of optical discs delaminate, oxidize, and develop "disc rot" that silently destroys sectors. Magnetic tape, the medium of choice for institutional cold storage, suffers from binder hydrolysis and print-through. Even the original ROM chips -- the mask ROMs and EPROMs on vintage arcade and cartridge PCBs -- are not immortal: EPROM charge dissipates over decades, and the ceramic and plastic packages that protect the silicon eventually admit moisture.

This is not a hypothetical future concern. It is happening now, continuously, to every copy of every file on every storage device in the world. The degradation is silent: filesystems do not report that a bit has flipped in the middle of a 512-byte sector. RAID arrays detect and repair some errors, but not all, and a rebuild after a drive failure can propagate latent corruption. A file that was correct when written may not be correct when read back three years later, and nothing in the operating system will volunteer this information.

The consequences for cultural preservation are stark. If you hold a collection of 50,000 ROM images on a NAS -- say, four 16 TB drives in RAID5 -- you are storing roughly 300 billion bits of data across physical media with documented unrecoverable bit error rates on the order of 10^-15 per bit read. Over a full scrub of the array, that arithmetic predicts a handful of undetected errors. Most will land in unoccupied space or be caught by filesystem checksums (if you are using ZFS or Btrfs -- ext4 and NTFS provide no data checksums at all). But some will land in file data and go unnoticed. Multiply this across years, across drive replacements, across migrations from one NAS to another, and the collection slowly, imperceptibly rots.

The only defense is an independent record of what the bits should be.

Checksums as the foundation of preservation

A cryptographic hash function takes an input of arbitrary length and produces a fixed-length output -- a fingerprint -- with the property that any change to the input, no matter how small, produces a completely different output. If you compute the SHA-1 hash of a 4 MB ROM file today and store that 160-bit number, then five years from now you can recompute the hash and compare. If the values match, the file is intact. If they differ, something has changed -- even if you cannot tell by looking at the file, even if the game still appears to run, a bit has flipped somewhere.

This is **fixity**: the assurance that a digital object has not been altered since a known-good reference point was established. It is the bedrock concept in digital preservation, shared by every organization that takes long-term data stewardship seriously -- the Library of Congress, the Internet Archive, the National Archives, the British Library, and (though they would not use the term) the volunteer communities that preserve video game software.

A **DAT file** is, at its core, a catalog of fixity assertions. For every known ROM image or disc dump in a given system's library, the DAT records the expected filename, the expected file size in bytes, and one or more cryptographic hashes computed when the data was known to be correct. The standard set is three algorithms:

- **CRC32** (32 bits): A cyclic redundancy check. Fast to compute, widely supported, used as the primary key for rapid file matching. Not cryptographically secure -- collisions are feasible to construct -- but adequate for initial identification.
- **MD5** (128 bits): A message digest algorithm. Faster than SHA-1, with a larger output space than CRC32. Theoretically broken for collision resistance since 2004 (Wang et al.), but preimage attacks remain computationally infeasible; MD5 is still useful for fixity verification where deliberate tampering is not the threat model.
- **SHA-1** (160 bits): The Secure Hash Algorithm. The standard confirmation hash in ROM preservation. Also theoretically weakened for collision resistance (SHAttered, 2017), but again, the threat here is accidental bitrot, not adversarial forgery, so SHA-1 remains perfectly adequate.

More recent DAT extensions add **SHA-256** and even SHA-384/SHA-512, though adoption is uneven. The principle is defense in depth: if three independent hash functions all agree, the file is almost certainly intact. If any one disagrees, corruption has occurred.

A ROM manager -- the software that consumes DAT files -- is therefore first and foremost a **fixity auditing tool**. When you "scan" a collection against a DAT, the tool computes hashes for every file on disk and compares them against the DAT's recorded values. The resulting report tells you which files are intact, which have been corrupted, which are missing entirely, and which files exist on disk but are unknown to the DAT. This operation is structurally identical to what the Library of Congress does when it runs BagIt fixity checks on its tape storage, or what LOCKSS nodes do when they poll each other for hash agreement. The scale differs; the principle is the same.

What the DAT knows that nobody has

Not every entry in a DAT file corresponds to a file that anyone actually possesses. This is one of the format's most important properties: **the catalog is more complete than any collection.**

Two mechanisms track different kinds of absence:

Nodump is the older concept, originating in the MAME community. When an arcade game's PCB contains a ROM chip that has been identified but not yet successfully read -- perhaps the chip is damaged, perhaps the decapping process needed to access a protected microcontroller has not been attempted, perhaps the specific PCB variant is exceedingly rare -- the corresponding `<rom>` entry in the DAT carries `status="nodump"`. No size or checksum is recorded because the data has never been seen. ROM managers treat nodump entries as expected absences rather than failures: they do not appear in missing-file reports, because there is nothing to find. Nodump is an assertion about hardware: *this chip exists on this board, but its contents are unknown.*

MIA (Missing In Action) is a different and more subtle concept, pioneered by No-Intro. An MIA entry describes a ROM that *has* been dumped at some point -- its checksums are known -- but for which no copy can currently be located in any known collection. The data was seen, verified, fingerprinted, and then lost: perhaps it existed in a single person's collection and was never widely distributed before that collection was lost to a drive failure. The DAT carries the full complement of hash values. The ROM manager can scan for it. But nobody has it.

The distinction matters for understanding preservation information flow. Nodump means the original extraction has never been performed -- the preservation process has not yet reached this artifact. MIA means the extraction succeeded, the data was characterized, but the preservation chain broke -- copies did not propagate widely enough to survive.

Redump takes a complementary approach to tracking completeness. Because optical disc preservation involves not just the data tracks but also physical media characteristics -- ring codes stamped into the inner hub, mastering SID codes, write offsets specific to each pressing plant -- Redump maintains **sidecar metadata** outside the main DAT file. The DAT itself carries track checksums for the data, but verification state (how many independent dumps have confirmed a disc, which physical ring code variants have been seen, which dumps used which drive and offset correction) is tracked in Redump's web database and is not fully representable in the Logiqx XML schema. This means Redump's DATs are necessary but not sufficient for full verification; the web database carries the provenance.

No-Intro's MIA tracking and Redump's sidecar verification represent two responses to the same underlying tension: the Logiqx XML DAT format was designed for fixity and identification, but preservation requires richer state than "file matches" or "file doesn't match." Both groups have extended the system in different directions -- No-Intro by adding status metadata to DAT entries, Redump by maintaining a parallel database -- because the core problem keeps revealing new edges. The data is always degrading. The question is how much context you carry about what you've verified, what you've lost, and what you never had.

ROM sets, auditing, and rebuilding

With the fixity foundation established, we can address the practical problem that originally motivated DAT file creation: **managing ROM collections across changing emulator versions.**

A **ROM set** is a collection of ROM image files -- binary data extracted from the read-only memory chips on physical game hardware -- packaged together so that an emulator can run a specific game. In the world of MAME (Multiple Arcade Machine Emulator), a single arcade game's ROM set is typically a ZIP file containing multiple individual ROM images, each corresponding to a different physical chip on the original printed circuit board. A complete MAME ROM collection as of version 0.286 (February 2026) encompasses roughly 48,900 machines and 364,800 individual ROM files, plus over 1,300 CHD disk images.

ROM sets change between emulator versions for several concrete reasons. Developers improve emulation accuracy, requiring additional data extracted from previously undumped chips -- sometimes obtained by physically decapping and photographing integrated circuits under a microscope. Previously corrupt or inaccurate dumps get replaced with better ones (a baddump becomes a good dump, the checksums change). ROM files are renamed as researchers discover the correct identities of original hardware components. Games are reclassified as parent/clone relationships shift. New machines are added. The result is that a ROM collection valid for MAME 0.260 may be subtly wrong for MAME 0.270: files renamed, files added, checksums changed.

Auditing a ROM set means computing CRC32, MD5, or SHA-1 checksums for each file on disk and comparing them against the values recorded in a DAT file. This is the fixity check described above, applied specifically to emulator compatibility. An audit report identifies missing ROMs, corrupt ROMs (checksum mismatch -- which could mean bitrot, could mean wrong version), misnamed ROMs, and unneeded extras. MAME itself includes a built-in audit via `mame -verifyroms`.

Rebuilding is the complementary operation: given a pool of ROM files from various sources -- an older set, a partial update, a grab bag of unknowns -- a rebuilder identifies files purely by their checksums and reorganizes them into the correct directory structure and filenames expected by the current emulator version. Because identification is content-based rather than name-based, a file named `garbage123.bin` can be correctly placed into the right game folder with the right name if its CRC32 matches a known ROM. This is checksums used not for fixity verification but for content-addressed identification -- the same hash serving two purposes.

By the late 1990s, MAME was growing rapidly -- adding dozens of new games with each release -- and managing thousands of ROM files manually was untenable. The community needed machine-readable catalogs: structured data files listing every game, its constituent ROM files, their sizes, and their checksums. These catalogs became the DAT files, and the tools that consumed them became ROM managers. But even though the immediate motivation was emulator housekeeping, the artifact they created -- a comprehensive, checksum-indexed catalog of every known ROM dump -- turned out to be something more valuable: a preservation manifest for an entire domain of cultural artifacts.

Three formats and a de facto standard: the history of DAT file specifications

The first ROM manager was **ClrMamePro**, created by **Roman Scherzer** and released in **1997** (https://mamedev.emulab.it/clrmamepro/). The name stands for "Cool Little ROM Manipulation and Management Engine." ClrMamePro defined a text-based DAT format called **ListInfo** (sometimes called the "CMPro format"), derived from MAME's own `-listinfo` output. It uses a nested parenthesis syntax:

```
clrmamepro (
    name "MAME"
    description "MAME 0.37b5"
)

game (
    name pacman
    cloneof puckman
    description "Pac-Man (Midway)"
    year 1980
    manufacturer "Midway"
    rom ( name pacman.6e size 4096 crc c1e6ab10 sha1 ... )
)
```

This format proved durable. ClrMamePro remains actively maintained as of 2025 (version 4.050), still Windows-only freeware, and the ListInfo format is still read by every major ROM manager.

A second format emerged from **RomCenter**, created by **Eric Bole-Feysot** (https://www.romcenter.com/). RomCenter used an INI-derived text format with sections like `[CREDITS]`, `[DAT]`, `[EMULATOR]`, and `[GAMES]`, using a special delimiter character to separate fields within game entries. RomCenter's format was less expressive than ClrMamePro's -- it lacked SHA-1 support and could not represent disk or sample entries -- but it served a large user base through the early 2000s. Bole-Feysot would later make a significant contribution to the preservation ecosystem by proposing the **1G1R (One Game, One ROM)** methodology on the No-Intro forums in 2008.

The format that ultimately prevailed was created by **Michael George**, known by the handle **Logiqx** (https://github.com/Logiqx). George, a British data specialist, developed a suite of command-line tools sharing a core library called **DatLib**: **DatUtil** (DAT creation and format conversion), **MAMEDiff** (DAT comparison between MAME versions), **ROMBuild** (ROM format conversion), **ROMInfo** (individual ROM identification), **ZIPIdent** (game identification), and **ImgChk** (image collection verification). All were written in C and shared the DatLib backend for loading, cleansing, converting, and saving DAT files across multiple formats.

On **March 17, 2008**, in DatLib version 2.23, George added the **"Generic XML"** format -- a clean, standardized XML schema for ROM management metadata. One week later, in DatLib v2.24 (March 24, 2008), he added a formal DTD (Document Type Definition) at revision 1.1. The DTD was refined over the following months, reaching its final form as **revision 1.5 on October 28, 2008**, with the public identifier:

```
PUBLIC "-//Logiqx//DTD ROM Management Datafile//EN"
SYSTEM "http://www.logiqx.com/Dats/datafile.dtd"
```

This format became the **de facto standard** for several reinforcing reasons. XML was universally parseable and self-documenting. Logiqx's DatUtil could convert freely among ClrMamePro ListInfo, RomCenter, MAME ListXML, and the new Generic XML, making it a lingua franca. All major ROM managers adopted it. Every major preservation group -- No-Intro, Redump, TOSEC -- chose it as their distribution format. And George himself actively curated and distributed high-quality DATs for dozens of emulators on his website, establishing the format through use as much as through specification.

A ghost standard that runs the world

George's website, http://www.logiqx.com/, has not been updated since approximately 2009 (ROMBuild v2.14, released December 19, 2009, was the last tool update). The domain eventually stopped resolving. The DTD URL that every Logiqx XML DAT file references in its DOCTYPE declaration -- `http://www.logiqx.com/Dats/datafile.dtd` -- returns nothing. The original DTD survives on the Internet Archive's Wayback Machine and in numerous GitHub repositories (e.g., https://github.com/flobot-oss/MvsxPackBuilder/blob/main/datafile.dtd). George himself remains active on GitHub (https://github.com/Logiqx) working on data analytics projects unrelated to ROM preservation.

Despite its creator's departure, the Logiqx XML format is more widely used than ever. The closest thing to a current maintainer of the extended specification is **SabreTools** (https://github.com/SabreTools/SabreTools), a comprehensive open-source DAT manipulation toolkit written in C# by **Matt Nadareski**. SabreTools reads and writes every major DAT format -- ClrMamePro, Logiqx XML, RomCenter, MAME ListXML, MAME Software Lists, DOSCenter, OfflineList, Archive.org format, hash files, and more. Its wiki (https://github.com/SabreTools/SabreTools/wiki/DatFile-Formats) constitutes the most comprehensive documentation of all DAT format variants currently available. SabreTools has extended the Logiqx schema to support SHA-256, SHA-384, SHA-512, and SpamSum hashes, though no formal updated DTD or XSD has been published. The format remains a community convention maintained through tool compatibility rather than a governed

standard.

MAME's own XML: ListXML and Software Lists

MAME generates its own XML output via `mame -listxml`, producing a comprehensive dump of all supported machines with embedded DTD. The output is enormous -- roughly 285 MB for a full MAME build -- and includes far richer data than a Logiqx DAT: chip definitions, display configurations, DIP switches, input mappings, driver status, device references, slots, and software list associations. The root element is `<mame>` (not `<datafile>`), and individual entries use `<machine>` (renamed from `<game>` in **MAME 0.162**, May 2015, when MAME and MESS merged). ROM managers can load this output directly as a DAT file, or tools like DatUtil can convert it to Logiqx XML. Pre-generated MAME DATs are available from **Progetto SNAPS** (https://www.progettosnaps.net/dats/MAME/).

MAME Software Lists are a separate but related system. These XML files, stored in the `hash/` directory of the MAME source tree (https://github.com/mamedev/mame/tree/master/hash), catalog known software for non-arcade systems: cartridges, floppy disks, tapes, and optical media for the hundreds of home computers and consoles MAME now emulates. Each file (e.g., `nes.xml`, `a7800.xml`, `ql_flop.xml`) uses its own DTD (`hash/softwarelist.dtd`) and describes software with metadata including name, description, year, publisher, ROM hashes (CRC32 and SHA-1), and part/dataarea structure reflecting the physical media organization. The Software Lists represent roughly 139,900 software items containing 236,900 ROM files and 11,400 disk images. Their stated philosophy -- preserving what was on the original media rather than any particular dump of it -- is itself a fixity assertion: the canonical data is defined by the content, not the container.

Anatomy of a Logiqx XML DAT file

A Logiqx XML DAT file has a clean, hierarchical structure. The root `<datafile>` element contains an optional `<header>` and one or more `<game>` (or `<machine>`) entries. The header carries catalog-level metadata: `<n>`, `<description>`, `<version>`, `<date>`, `<author>`, and optional elements for `<email>`, `<homepage>`, `<url>`, and `<comment>`. Two special empty elements -- `<clrmamepro>` and `<romcenter>` -- embed tool-specific hints about merge policy, packing format, and nodump handling.

Each `<game>` element has a required `name` attribute (used as the ZIP filename) and optional attributes encoding relationships: **`cloneof`** names the parent game (establishing that this entry is a variant -- a regional release, a revision, a bootleg), **`romof`** names a ROM parent (typically a shared BIOS set like `neogeo`), and **`sampleof`** names a sample audio parent. The `isbios` attribute flags BIOS sets.

Within each game, `<rom>` elements describe individual ROM files with:

- **`name`**: the filename within the ZIP archive.
- **`size`**: file size in bytes.
- **`crc`**: CRC32 as a hexadecimal string -- the fast-match key.
- **`md5`**: MD5 hash -- the intermediate verification layer.

- `sha1`: SHA-1 hash -- the primary fixity check.
- `status`: one of four values:
  - `good` (default): verified correct dump, checksums match original hardware data.
  - `baddump`: known to be a corrupt or inaccurate dump. The checksums record *what was dumped*, not what the original data should be. These entries are preserved because even a bad dump is evidence.
  - `nodump`: the ROM chip exists on original hardware but has not been successfully extracted. No checksums are recorded.
  - `verified`: independently confirmed by multiple sources (used by some DAT producers to distinguish single-source and multi-source verification).
- `merge`: indicates that this ROM should be shared with a parent set in split or merged configurations.

`<disk>` elements describe CHD files similarly but carry SHA-1 only (CHDs embed their own internal checksums and do not use CRC32 at the container level).

Extensions adopted since the original 2008 DTD include SHA-256 hashes, **serial number fields** (used by Redump and No-Intro for physical media identification -- a bridge between the digital fixity world and the physical artifact), the **MIA flag** (described above), and `<release>` elements encoding region, language, and date metadata critical for 1G1R filtering. These extensions have been adopted through tool consensus rather than formal schema revision -- a pattern that has worked well enough for two decades but that creates interoperability risks as the ecosystem grows.

Part 2: The preservation groups

No-Intro: the clean-dump purists

No-Intro (https://no-intro.org) emerged around **2003--2004** from the Game Boy Advance ROM scene. The name is a direct reference to the practice of removing "intros" -- the short animated cracktro sequences that warez scene groups would hack into GBA ROMs before distribution to advertise their releases. No-Intro's founding mission was to catalog only clean, unmodified ROM dumps: no intros, no hacks, no trainers, no overdumps, no underdumps. Just the closest possible representation of what was on the original cartridge.

The project is an **informal volunteer community**, not a legal entity. Key long-standing contributors include **xuom2** (database and DAT generation) and **kazumi213** (parent/clone lists, SNES work). No-Intro explicitly states: "We are NOT in the Console Scene and we are not involved in any P2P sharing."

No-Intro's primary interface is **DAT-o-MATIC** (https://datomatic.no-intro.org), a web-based database that allows users to download DAT files for individual systems or daily bulk packs, search the database by game title, and check dump verification status. ROMs are categorized as **Verified** (two or more independent trusted dumps match), **Not Verified** (single trusted dump),

Bad (known corrupt), or **MIA** (not found on the internet). DATs are distributed in Logiqx XML format. The project also maintains a wiki (https://wiki.no-intro.org), forums (https://forum.no-intro.org), and a Discord server.

No-Intro's scope centers on **cartridge-based systems** -- NES, SNES, N64, Game Boy family, GBA, DS, 3DS, Sega Genesis, Master System, Game Gear, Atari consoles, TurboGrafx-16, Neo Geo Pocket, and dozens more -- but has expanded to cover digital download content and some extracted disc-based data. The project stores metadata and hashes for over **300,000 items** across more than 100 systems. Some DATs for modern or current-generation systems are kept "private" to minimize friction with copyright holders.

No-Intro's **parent/clone methodology** borrows from MAME: different regional releases and revisions of the same game are grouped, with one designated parent and others as clones. Parent selection follows a convention: final releases over prototypes, English-containing versions preferred, World releases over regional ones, and highest revision numbers. The **1G1R** concept, proposed by Eric Bole-Feysot in 2008, allows users to keep only one version of each game based on region and language preferences. Implementation uses `<release>` tags in DAT entries encoding region and language, which ROM managers can filter against a user-defined priority list.

No-Intro and Redump are **complementary partners**: No-Intro handles cartridge and ROM media; Redump handles optical discs. They share naming convention influences, and tools like Retool work with both.

Redump: forensic-grade optical disc preservation

Redump (http://redump.org) was founded around **2006** -- a 10th-anniversary celebration circa 2016 confirms the timeline. The name embodies the project's philosophy: previous disc dumps were insufficiently accurate and needed to be done again ("re-dumped") with better methods. Key administrators include **iR0b0t** (primary database admin) and **F1ReB4LL**.

Redump's scope is **all optical media containing video game data**: "If it spins, is read by a laser, and has digital game data on it, we want to preserve it." This includes CDs, DVDs, GD-ROMs (Dreamcast), UMDs (PSP), Blu-ray discs, and HD-DVDs across PlayStation 1/2/3/4, Sega Saturn, Dreamcast, GameCube, Wii, Xbox, Xbox 360, PC, Mac, 3DO, CD-i, Neo Geo CD, and many more. The database contains over **50,000 PC discs alone**, with the total catalog substantially larger.

Redump's **dumping standards** are the most technically demanding in the preservation community. The preferred drives are **Plextor** models with Sanyo chipsets (PX-760A, PX-755, PX-716, Premium, Premium2, and others), because they support the **D8 vendor read command** -- a legacy SCSI command that reads CD sectors in a raw, scrambled mode. This is critical because it allows detection of the **combined read/write offset**: the sum of the drive's inherent read offset and the disc's write offset from the mastering process. Correct offset compensation is essential for producing bit-perfect dumps. Plextors also support lead-in/lead-out reading, reliable C2 error reporting, Data Position Measurement (for copy protection detection), and subchannel data extraction. The **PX-760A is the last true Plextor** supporting D8; later "Plextor" branded drives are rebranded units without these capabilities. Recent developments have expanded support to ASUS BW-16D1HT drives with MediaTek chipsets, especially via custom firmware by RibShark.

The primary dumping tools are **DiscImageCreator** (DIC), created by **sarami** (https://github.com/saramibreak/DiscImageCreator), a command-line tool active since circa 2012, and the newer **redumper** by **superg** (https://github.com/superg/redumper), accepted for Redump submissions starting 2023. **Media Preservation Frontend** (MPF, https://github.com/SabreTools/MPF), a GUI wrapper for both tools created by the SabreTools team, simplifies the submission process.

Verification requires **two independent dumps from different people** to match. Redump collects extensive metadata per disc: title, serial numbers, barcodes, ring codes (mastering SID, mould SID, toolstamp), region, languages, version, build date, write offset, error counts, track layouts, and per-track checksums (CRC32, MD5, SHA-1). DATs are downloadable from http://redump.org/downloads/ in standard XML format. Redump does not host disc images -- only metadata and checksums.

TOSEC: the universal cataloger

TOSEC (The Old School Emulation Center, https://www.tosecdev.org) was founded in **February 2000**, making it the oldest of the three major cataloging groups. Unlike No-Intro (which catalogs only verified clean dumps) or Redump (which demands forensic-grade disc images), TOSEC aims to **catalog everything**: good dumps, bad dumps, hacks, translations, cracked versions, overdumps, homebrew, magazine cover disks, scanned publications, box art, and video advertisements. This makes it simultaneously the most comprehensive historical record and the least curated for practical use.

TOSEC covers an extraordinary breadth: over **195 unique brands** and hundreds of computing platforms, with more than **1.2 million cataloged software images** comprising approximately **8 TB** of data. Over half of all entries are for Commodore platforms (Amiga, C64, C128, VIC-20). The project also catalogs arcade machines, early minicomputers, and non-software materials like magazines and product photography.

TOSEC's most distinctive contribution is the **TOSEC Naming Convention (TNC)**, a structured filename format encoding title, version, demo status, date, publisher, system, country (ISO 3166-1 alpha-2), language (ISO 639-1), copyright status, development status, media type, and dump-quality flags in square brackets (`[cr]` for cracked, `[f]` for fixed, `[h]` for hacked, `[b]` for bad dump, `[!]` for verified). Example: `Legend of TOSEC, The (1986)(Devstudio)(US)[cr PDX][h TRSi]`. The TNC specification was first formalized on July 20, 2008, with updates in 2011 and the latest revision on March 23, 2015.

DAT packs -- complete ZIP archives of all DATs -- are released approximately twice yearly from https://www.tosecdev.org/downloads, with the most recent stable release being TOSEC-v2024-05-17 containing nearly 4,000 individual DAT files.

MAME: the emulator that is also a database

MAME (https://www.mamedev.org, source at https://github.com/mamedev/mame) was first released on **February 5, 1997** by **Nicola Salmoria**. Its stated purpose is to preserve gaming history by preventing vintage games from being lost; the ability to play them is described as a "beneficial side effect." MAME became open source under **GPL-2.0+** on March 4, 2016 (version

0.172). It is a registered trademark of Gregory Ember.

MAME functions as both emulator and preservation database. The `mame -listxml` command outputs comprehensive XML for every supported machine, serving as a de facto DAT file. MAME's catalog encompasses three distinct set types. **Non-merged sets** are fully self-contained: every ZIP includes all ROMs needed to run that game, including BIOS and shared device files. **Split sets** separate parent and clone data: clones contain only files differing from the parent and require the parent ZIP to be present. **Merged sets** combine all parent and clone ROMs into a single archive -- the most space-efficient format. The choice among these is a user preference managed by ROM managers.

The **MESS** (Multi Emulator Super System) project, which emulated home computers and consoles, **merged into MAME with version 0.162 on May 27, 2015**, led by developer **Miodrag Milanovic**. This made MAME a single unified project covering arcade machines and home systems alike.

CHD (Compressed Hunks of Data) is MAME's lossless compression format for large media, created by **Aaron Giles** and introduced in **MAME v0.59 (March 2002)**. Originally "Compressed Hard Disk," it was generalized as arcade machines increasingly used hard drives and optical media. CHD breaks data into fixed-size "hunks" compressed independently with LZMA, zlib, Huffman, or FLAC. The `chdman` tool (https://docs.mamedev.org/tools/chdman.html) manages CHD creation, verification, and extraction. CHD has been adopted beyond MAME by RetroArch, DuckStation, and other emulators via the standalone **libchdr** library (https://github.com/rtissera/libchdr).

GoodTools: the fallen standard

GoodTools were a suite of **35 ROM auditing applications** created by a developer known only as **"Cowering"** (real name unknown), active from the late 1990s through 2016. Each tool -- GoodNES, GoodSNES, GoodGBA, GoodGen, and so on -- contained an embedded database of known ROM hashes for a specific platform. GoodTools scanned collections and renamed files using a bracket-code convention that became deeply ingrained in ROM culture: `[!]` for verified good dump, `[b]` for bad, `[h]` for hack, `[o]` for overdump, `[T]` for translation, `(U)` for USA, `(J)` for Japan.

GoodTools fell out of favor for several reasons: the hash databases were closed and opaque (no public verification system), the sets mixed clean dumps with hacks, trainers, and bad data, the tools could not export standard DAT files for use with ROM managers, and Cowering eventually stopped maintaining them. The last known release was **April 3, 2016**. As one No-Intro community member noted: "Without a public and documented verification system, we can't carry over ANY of his verifications."

The **OpenGood** project (https://github.com/SnowflakePowered/opengood) preserves freely available XML DAT files for ROMs listed in the final GoodTools release, created for historical archival purposes. The project explicitly states: "OpenGood does not encourage the continued use of GoodTools. No-Intro, Redump, and TOSEC have done a much better job."

Myrient: a preservation mirror approaches its end

Myrient (https://myrient.erista.me/) launched in **October 2022** as a service of **Erista** (https://erista.me/), operated by a single individual known as **Alexey**. The name is a portmanteau of "myriad" and "entertainment." At its peak, Myrient hosted over **390 TB** of curated, verified ROM and disc image collections organized by No-Intro, Redump, TOSEC, MAME, and FinalBurn Neo DAT standards, all verified by hash comparison against canonical DATs and stored in TorrentZip-formatted ZIP archives. Access was free -- no login, no ads, no paywall -- making it the largest publicly accessible ROM distribution service.

On **February 26, 2026**, Alexey announced that Myrient would shut down on **March 31, 2026**, citing unsustainable costs -- he was paying over **$6,000 per month** out of pocket to cover the shortfall between donations and hosting expenses, exacerbated by rising hardware costs driven by AI datacenter demand. The community responded rapidly: the **Minerva Archive** project backed up approximately 385 TB of data through volunteer effort, and Jason Scott of the Internet Archive cautioned the community against indiscriminate mass uploads, noting that very little of Myrient's content was truly unique to that mirror.

The Software Preservation Society: reading magnetic ghosts

The **Software Preservation Society (SPS)** (http://www.softpres.org), formerly the **Classic Amiga Preservation Society (CAPS)**, was **founded in 2001** by **Istvan Fabian**, a Hungarian computer expert. SPS focuses on preserving software distributed on **magnetic media** -- primarily Amiga, Commodore 64, Atari ST, ZX Spectrum, and Amstrad CPC floppy disks -- where the physical medium is actively degrading as magnetic particles lose their orientation or detach from the substrate.

SPS's most significant contribution is the **KryoFlux** (https://kryoflux.com/), a USB-based hardware device that reads **flux transitions** from floppy disks at extremely fine temporal resolution, capturing not just the data but the analog signal characteristics that encode copy protection schemes and timing-dependent formats. KryoFlux has become the **gold standard for floppy disk archival** at memory institutions worldwide. Their **IPF (Interchangeable Preservation Format)** captures disk structure and copy protection metadata, not just sector data, making it the de facto standard for authentic Amiga disk images. Preserved images are not made publicly available; contributors who send in original disks receive preserved images in return.

The **Software Preservation Network (SPN)** (https://www.softwarepreservationnetwork.org/) is a separate, US-based academic organization founded in **2016** by **Jessica Meyerson** (UT-Austin) and **Zach Vowell** (Cal Poly), hosted by the Educopia Institute. SPN's most significant achievement is securing **DMCA Section 1201 exemptions** for software preservation through the Library of Congress's triennial rulemaking process, represented by the Harvard Law School Cyberlaw Clinic. The exemption, first granted in **October 2018**, allows libraries, archives, and museums to circumvent DRM on lawfully acquired software for preservation purposes, though with restrictions including a physical-premises requirement that the community has been fighting to remove.

Internet Archive: the library that plays the games

The **Internet Archive** (https://archive.org/), the nonprofit digital library founded by Brewster Kahle, plays a massive role in game preservation. Its **Emularity** system, running since 2013, uses JavaScript/WebAssembly ports of MAME, DOSBox, and other emulators to offer over **250,000 emulated software items** playable directly in the browser. Key collections include **The Internet Arcade** (vintage arcade games), the **Console Living Room** (home console games), and the **MS-DOS Software Library**. **Jason Scott** (https://mastodon.archive.org/@textfiles), who joined the Archive in 2011 as "Free-Range Archivist" and Software Curator, has been the driving force behind these initiatives, guided by the philosophy that "access drives preservation."

Every item on Archive.org has machine-readable metadata: `<identifier>_meta.xml` for item-level data and `<identifier>_files.xml` for per-file checksums (MD5, CRC32, SHA-1). The Metadata API at `https://archive.org/metadata/<identifier>` returns this as JSON, making it programmatically accessible. Community members regularly upload No-Intro, Redump, and MAME sets, and Hidden Palace hosts prototypes there.

The Archive faces legal pressure. The **Hachette v. Internet Archive** lawsuit (filed June 2020, Second Circuit ruling September 4, 2024) found the Archive liable for copyright infringement in its book lending program, rejecting the Controlled Digital Lending theory. While this case concerned books rather than games, it has implications for digital preservation legal theory. The Archive's software collections operate under different frameworks, including Section 108 library exceptions and DMCA exemptions.

Hidden Palace: rescuing what was never released

Hidden Palace (https://hiddenpalace.org/) was founded around **2005** by **drx**, with roots in the Sonic the Hedgehog hacking and prototype community. The site, relaunched as a MediaWiki in 2015 for its 10th anniversary, is dedicated to preserving video game development materials: prototypes, pre-release builds, source code, and design documents.

Hidden Palace's largest initiative is **Project Deluge**, launched March 20, 2021, which systematically archives a massive lot of development media obtained through an anonymous collector connected via Jason Scott. Releases have included over **750 unique PS2 prototypes** (including E3 demos of Shadow of the Colossus and God of War 2), hundreds of Xbox, Dreamcast, PlayStation, and Saturn prototypes, 207 Xbox 360 prototypes, and 114 Wii prototypes -- nearly **5,000 discs** processed in total. The project continues with regular releases through 2026, including prototypes of Fallout: New Vegas, Dead Rising, and Assassin's Creed.

Other notable groups and projects

TruRip/EmuArc (https://database.trurip.org/) is a specialist optical disc preservation project emphasizing maximum fidelity including subchannel data; it has rebranded from TruRip to EmuArc but remains semi-private with limited DAT accessibility. **Pleasuredome** was a major private BitTorrent-based MAME ROM distribution community that operated for over 15 years before shutting down in **September 2021**; its guides and DAT file resources survive at https://pleasuredome.github.io/pleasuredome/ and https://pleasuredome.miraheze.org/wiki/Main_Page. **DATVault** (https://www.datvault.com/) is a DAT delivery service integrated into RomVault, offering automatic DAT updates -- free for MAME, Patreon-supported for No-Intro, Redump, and

TOSEC DATs. The **libretro/RetroArch database** (https://github.com/libretro/libretro-database) bulk-imports DATs from No-Intro, Redump, and TOSEC, compiling them into RetroArch's proprietary `.rdb` format for game identification, metadata enrichment, and playlist generation.

Part 3: The ROM manager ecosystem

How ROM management actually works

The workflow is straightforward in concept: obtain a DAT file from a preservation group, load it into a ROM manager to create a "profile," scan your existing ROM files against the profile (the tool computes checksums and compares), review the audit report (missing, extra, misnamed, corrupt files), and fix or rebuild the collection. The insight that makes this work is that **identification is by content, not by name**: a ROM manager matches files by their CRC32 (fast, 32-bit), confirmed by SHA-1 (160-bit, cryptographically strong), regardless of what the file is currently called.

Rebuilding is file-based: the tool scans all files in a source directory (including inside archives), computes CRC32 for each, matches against the loaded DAT, and creates correctly named files in the destination. **Auditing** is the read-only complement: scan, checksum, compare, report.

The tools

ClrMamePro (https://mamedev.emulab.it/clrmamepro/), created by Roman Scherzer in 1997, remains the most respected single-DAT ROM manager after **28 years** of continuous development. Version 4.050 was released in 2025. It is Windows-only freeware (usable on Linux via Wine). Its strength is deep, precise control over scanning, rebuilding, and merging operations, with support for ZIP, 7z, and RAR archives plus CHD verification. Its limitation is that it processes one DAT profile at a time.

RomVault (https://www.romvault.com/), created by **GordonJ** starting around 2005, addresses ClrMamePro's single-DAT limitation with **multi-DAT simultaneous processing** -- it can load thousands of DAT files at once and process them together. Cross-platform via .NET/Mono, it uses a tree-based directory structure for organizing DATs and ROM collections. Version 3.7.0 (April 2024) introduced the RVZSTD compression format. RomVault integrates with DATVault for automatic DAT updates and has an active Discord community.

Igir (https://igir.io/, https://github.com/emmercm/igir), pronounced "eager," is a modern CLI tool created by **Christian Emmer** in Node.js/TypeScript, approaching its third birthday as of August 2025. Its philosophy is "zero setup" -- it works with minimal configuration, supports multiple simultaneous operations (copy, move, link, extract, zip, test, clean), produces **TorrentZip by default** since v4.0.0, and includes built-in output tokens for organizing ROMs for specific devices and frontends including MiSTer FPGA, Analogue Pocket, Batocera, RetroDECK, and EmulationStation. Igir's documentation at https://igir.io/misc/torrentzip/ contains one of the most complete public specifications of the TorrentZip format.

SabreTools (https://github.com/SabreTools/SabreTools), by Matt Nadareski, is not a traditional ROM manager but a **DAT manipulation Swiss army knife**: format conversion, DAT merging/splitting, filtering, statistics, header management, and verification. It reads and writes more DAT formats than any other tool. The broader SabreTools ecosystem includes **MPF** (Media Preservation Frontend, https://github.com/SabreTools/MPF) for disc dumping, **BinaryObjectScanner** for copy protection detection, and **NDecrypt** for cartridge encryption handling.

oxyROMon (https://github.com/alucryd/oxyromon), by Maxime Gauduin, is a Rust-based CLI organizer emphasizing archival purity -- it only supports original and lossless formats natively, with lossy export as a separate operation. It features a SQLite database backend, web UI, and automatic Redump DAT downloading. **Romulus** (https://romulus.dats.site/), by F0XHOUND, was a beginner-friendly Windows ROM manager now discontinued. **Retool** (https://github.com/unexpectedpanda/retool), by unexpectedpanda, was a specialized DAT filter providing superior 1G1R filtering logic compared to built-in ROM manager implementations; it has been discontinued but continues as **Retool-Redux** (https://github.com/coreyemtp/retool-redux).

TorrentZip: deterministic compression for distributed verification

TorrentZip solves a subtle but critical problem. When ROM sets are distributed via BitTorrent, files are verified by their hashes. If two people create ZIP archives of identical ROM files using different tools, the resulting archives will differ -- different timestamps, compression parameters, file ordering, internal metadata -- causing BitTorrent hash mismatches. TorrentZip specifies rules for **deterministic ZIP creation**: ZLib maximum compression (level 9), all file timestamps set to **11:32 PM, December 24, 1996** (the date of MAME's first public release), files sorted by lowercase filename, forward slashes only, no data descriptors, and a validation checksum in the ZIP comment field (`TORRENTZIPPED-XXXXXXXX` where the Xs are a CRC32 of the central directory records). The result: anyone running TorrentZip on identical input files produces a bit-identical output ZIP.

The original `trrntzip` tool (https://sourceforge.net/projects/trrntzip/) emerged from the Pleasuredome community. GordonJ's .NET implementation is available at https://www.romvault.com/trrntzip/, and a Go implementation exists at https://github.com/uwedeportivo/torrentzip. All major ROM managers now produce TorrentZip output natively. The specification document is available at https://www.romvault.com/trrntzip_explained.pdf.

Part 4: The library science connection

DAT files as shadow OAIS implementations

The **Open Archival Information System** (OAIS, ISO 14721), developed by the Consultative Committee for Space Data Systems and revised in 2012, is the foundational conceptual framework for digital preservation. It defines three information package types that map surprisingly well onto ROM preservation practice.

The **Submission Information Package (SIP)** -- content as received from the producer -- corresponds to the initial ROM dump from physical media: the raw binary plus whatever metadata the dumper provides (dump date, hardware used, initial checksums). The **Archival Information Package (AIP)** -- the canonical stored package -- corresponds to the verified ROM file together with its entry in the canonical DAT: the checksum-validated binary linked to structured metadata (game name, region, revision, platform, publisher, year). The **Dissemination Information Package (DIP)** -- the package delivered to users -- corresponds to the ROM file as reorganized by a ROM manager for use with a specific emulator, potentially renamed, recompressed, or restructured (merged vs. split vs. non-merged).

OAIS's **Preservation Description Information** includes Reference Information (identifiers), Provenance (origin and history), Context (relationships), and Fixity (checksums). ROM DATs excel at fixity and reference information but are weaker on provenance -- dump source, date, method, and hardware are sometimes tracked internally by No-Intro and Redump but rarely appear in public DAT files. Jerome McDonough's seminal paper "Packaging Videogames for Long-Term Preservation: Integrating FRBR and the OAIS Reference Model" (*JASIST*, 62(1), 171--184, 2011) is the most rigorous academic treatment of applying OAIS to video game objects.

Checksums everywhere: BagIt manifests and DAT files as structural cousins

The **BagIt** specification (RFC 8493, https://datatracker.ietf.org/doc/html/rfc8493), jointly developed by the Library of Congress and the California Digital Library, defines a hierarchical file packaging format built around **manifest files** that list each payload file's checksum. A BagIt manifest is structurally a DAT file stripped to its essence: one line per file, hash plus path, enabling fixity verification at ingest and throughout the preservation lifecycle.

The Library of Congress built its entire Content Transfer System around BagIt, verifying content at every step from creation through delivery to storage (https://blogs.loc.gov/thesignal/2019/04/bagit-at-the-library-of-congress/). ROM DATs serve the same function at a different scale: where a BagIt manifest covers one bag (one accession), a ROM DAT covers an entire platform's known catalog -- potentially tens of thousands of entries.

ROM DATs typically include **three independent hash algorithms** per file (CRC32, MD5, SHA-1), while most institutional systems use one or two. The community's practice of multi-algorithm verification provides defense-in-depth that institutional preservationists could learn from. Conversely, institutional tools like **JHOVE** (https://openpreservation.org), **FITS**, and **DROID** perform format identification and validation that ROM tools do not attempt -- they assume format knowledge is implicit in the platform designation.

PREMIS metadata: where the gaps are widest

PREMIS (Preservation Metadata: Implementation Strategies, https://www.loc.gov/standards/premis/v3/) is the de facto international standard for preservation metadata, maintained by the Library of Congress since 2005. PREMIS defines five entities: Intellectual Entities, Objects, Events, Agents, and Rights. The comparison with ROM DATs reveals both strong overlaps and critical gaps.

The overlaps center on **fixity**: PREMIS records message digests (checksums), the algorithm used, and the generating software, exactly as ROM DATs do. Object size is directly represented in both. Intellectual entity metadata (game name, region, revision) maps well to PREMIS's descriptive framework.

The gaps are significant. PREMIS's **Events entity** tracks every action performed on an object -- virus scans, format migrations, fixity checks, ingest events -- with timestamps and responsible agents. ROM DATs record final state only; there is no standardized way to express "this ROM was dumped on March 3, 2019, using a Retrode 2, by user xyz, and independently verified by user abc on April 7, 2019." PREMIS **Rights entities** explicitly model copyright status, license terms, and access restrictions; ROM DATs contain no rights information whatsoever. **Provenance** -- the chain of custody from physical media through dump process to verified archive -- is at best partially captured in No-Intro's internal databases and rarely appears in public DATs.

LOCKSS and the accidental distributed archive

LOCKSS (Lots of Copies Keep Stuff Safe, https://www.lockss.org/), developed at Stanford by Vicky Reich and David Rosenthal in 1999, is an open-source peer-to-peer digital preservation system. Its core principles -- distributed replication across independent nodes, integrity validation via cryptographic hash polling, consensus-based repair of damaged copies, no single point of failure -- describe the ROM preservation community's distribution model with eerie precision.

The community maintains ROM sets across mirror sites, torrent swarms, personal collections, and institutional archives like the Internet Archive. Any user can verify their local copy against the canonical DAT using standard tools. When corruption is detected, re-dumps and re-verification occur. The distribution is global and decentralized. The parallel is structural, not deliberate: the ROM community independently reinvented LOCKSS principles without knowledge of the formal system.

The critical difference is **governance**. LOCKSS operates with publisher permission within copyright law; ROM distribution exists largely outside formal legal authorization. LOCKSS includes automated repair mechanisms; the ROM community relies on manual re-dumping. LOCKSS has formal succession planning; ROM preservation groups depend on a few key volunteers and could disappear if maintainers leave, as Myrient's shutdown vividly demonstrated.

What each side can teach the other

The Library of Congress holds over **5,000 video games** (https://www.loc.gov/item/webcast-9230/) and publishes a Recommended Formats Statement for software and video games (https://www.loc.gov/preservation/resources/rfs/software-videogames.html). The **NDSA Levels of Digital Preservation** (https://www.ndsa.org/publications/levels-of-digital-preservation/) provide a maturity framework. The **Video Game History Foundation** (https://gamehistory.org/), founded in 2016 by **Frank Cifaldi**, operates the first dedicated video game history research library in the US and published a landmark **2023 study** (with SPN) finding that **87% of games released before 2010 are not commercially available** -- they are effectively "critically endangered." The academic journal **ROMchip** (https://romchip.org/), co-founded by Henry Lowood (Stanford), Laine Nooney (NYU), and Raiford Guins (Indiana University), publishes peer-reviewed game history

research.

Institutional preservationists can learn from the ROM community's **scale of distributed verification** (thousands of volunteers, hundreds of thousands of verified items), their **multi-algorithm fixity practice**, their **completeness of cataloging** (every known regional variant and revision systematically documented), and their **deep practical knowledge** of hardware quirks, copy protection, and format-specific preservation challenges. The community's DATs are updated daily via No-Intro's DAT-o-MATIC; institutional metadata updates often lag by months.

The ROM community can learn from institutional preservation's **provenance tracking** (PREMIS events modeling), **rights management** frameworks, **long-term sustainability planning** (NDSA Levels, succession planning, disaster recovery), and **standardized metadata schemas** that enable interoperability across institutions. Mapping DAT metadata to PREMIS or Dublin Core would open doors to institutional collaboration. Structuring preservation workflows around explicit OAIS SIP/AIP/DIP transformations would provide a trust framework for institutional partners. The Preserving Virtual Worlds project (McDonough et al., 2010, funded by LoC's NDIIPP) represents the most significant academic attempt to bridge these worlds, creating an OWL ontology integrating OAIS and FRBR for game objects.

Timeline of key dates

Year	Event
1997	MAME 0.1 released (Feb 5) by Nicola Salmoria; ClrMamePro 1.0 released by Roman Scherzer
Late 1990s	RomCenter released by Eric Bole-Feysot; GoodTools suite active under Cowering
2000	TOSEC founded (February)
2001	Software Preservation Society founded as CAPS by Istvan Fabian
2002	CHD format introduced in MAME v0.59 (March) by Aaron Giles
2003--2004	No-Intro founded, emerging from GBA ROM scene
~2005	Hidden Palace founded by drx; RomVault development begins
2006	Redump founded
2008	Logiqx XML "Generic XML" format added to DatLib v2.23 (March 17); DTD formalized (March 24); DTD reaches final revision 1.5 (October 28); 1G1R concept proposed by Eric Bole-Feysot on No-Intro forums; TOSEC Naming Convention first formalized (July 20); first DMCA game preservation exemption (games requiring defunct servers)

2009	Last update to Logiqx tools (ROMBuild v2.14, December 19)
~2010	logiqx.com effectively goes dark
2012	DiscImageCreator first appears in Redump forums
2013	Internet Archive's Emularity browser emulation system launches; TOSEC files begin inclusion in Internet Archive
2015	MAME and MESS merge in version 0.162 (May 27); Hidden Palace wiki relaunched
2016	MAME goes open source under GPL-2.0+ (March); Video Game History Foundation founded by Frank Cifaldi; Software Preservation Network founded; last GoodTools release (April 3); Cowering's last blog post
2018	DMCA Section 1201 exemption for software preservation secured by SPN and Harvard Cyberlaw Clinic (October 26)
2019	NDSA Levels of Digital Preservation v2.0 published
2021	Hidden Palace Project Deluge launched (March); Pleasuredome shuts down (September)
2022	Myrient launched (October) by Alexey/Erista; Igir first released (August)
2023	Redumper accepted for Redump submissions; VGHF/SPN "87% study" published; Hachette v. Internet Archive district court ruling (March)
2024	Hachette v. Internet Archive Second Circuit ruling affirmed (September 4); DMCA 2024 rulemaking denies remote access for game preservation; RomVault v3.7.0 introduces RVZSTD (April)
2025	VGHF Digital Library launches (January) with 30,000+ files; Retool discontinued
2026	Myrient announces shutdown (February 26), effective March 31; Minerva Archive backs up 385 TB; MAME reaches v0.286 (February)

Glossary

1G1R (One Game, One ROM): A filtering methodology that selects one preferred version of each game from all known regional variants and revisions, based on user-defined region and language priorities.

AIP (Archival Information Package): In OAIS, the package stored within the archive system, containing the content object and its associated preservation metadata.

Baddump: A ROM image known to be corrupt, incomplete, or inaccurately dumped. Flagged with `status="baddump"` in DAT files; these ROMs do not match the original hardware data.

CHD (Compressed Hunks of Data): MAME's lossless compression format for large media (hard drives, CD-ROMs, LaserDiscs). Data is divided into fixed-size "hunks" compressed independently. Created by Aaron Giles in 2002.

Clone: A variant of a parent game: a regional release, revision, bootleg, or prototype. In split/merged ROM sets, clones share common data with the parent to save space.

CRC32: A 32-bit cyclic redundancy check used as the primary fast-matching hash in ROM identification. Quick to compute but not cryptographically secure.

DAT (Datafile): A machine-readable catalog listing every known ROM or disc image for a given system or emulator, including filenames, sizes, and cryptographic checksums. Typically in Logiqx XML or ClrMamePro ListInfo format.

Fixity: In digital preservation, the property that a digital object has not been altered. Verified through checksum comparison.

Merged set: A ROM set configuration where the parent ZIP contains all ROM files for itself and all its clones combined. Most space-efficient.

MIA (Missing In Action): A ROM or game known to have been dumped at some point -- its checksums exist in the DAT -- but for which no copy can currently be located in any known collection. The preservation chain broke before the data propagated widely enough to survive.

Nodump: A ROM chip known to exist on original hardware but not yet successfully read/extracted. No checksums are recorded. Flagged with `status="nodump"` in DAT files; ROM managers do not report these as "missing."

Non-merged set: A ROM set where every ZIP is fully self-contained, including all parent, BIOS, and device ROMs needed to run independently. Most space-inefficient but most portable.

OAIS (Open Archival Information System): ISO 14721, the foundational reference model for digital archives, defining information packages (SIP, AIP, DIP), functional entities, and preservation concepts.

Parent: The primary version of a game in a parent/clone relationship, typically the original release or most complete version.

PREMIS: Preservation Metadata: Implementation Strategies, the de facto international standard for preservation metadata, maintained by the Library of Congress.

ROM (Read-Only Memory): In preservation context, a binary image of the data stored on a read-only memory chip or cartridge. By extension, any binary dump of game software from original media.

ROM set: The complete collection of ROM files needed by an emulator to run a specific game, typically packaged as a ZIP file named after the game's short identifier.

Split set: A ROM set where parent ZIPs contain parent-specific files and clone ZIPs contain only files that differ from the parent. Clones require the parent to be present.

TorrentZip: A specification for deterministic ZIP compression ensuring identical input files always produce bit-identical output archives, enabling reliable BitTorrent-based distribution and verification.